
ECE 726 — Assignment 3

Neural Networks from Scratch

Fashion MNIST Multi-Class Classification

Student: Mahan Choudhury

Student ID: 400648097

Course: ECE 726 — Machine Learning, Winter 2026

Instructor: Dr. Sorina Dumitrescu

Department: Electrical & Computer Engineering, McMaster University

All neural network operations — forward pass, backpropagation, and optimizers — are implemented from scratch using NumPy only.

Contents

1	Introduction	2
2	Methodology	2
2.1	Data Preprocessing	2
2.2	Network Architecture	2
2.3	Loss Function	3
2.4	Backpropagation	3
2.5	Training Algorithms	3
2.6	Regularization and Early Stopping	4
2.7	Hyperparameters	4
3	Results — Main Experiment	4
3.1	Learning Curves	4
3.2	Misclassification Errors	7
3.3	Summary Visualizations	8
4	Discussion — Main Experiment	9
4.1	Adam-Stochastic Divergence	9
4.2	Impact of Network Depth (Fixed Optimizer)	10
4.3	Impact of Optimizer (Fixed Architecture)	10
4.4	Generalization Gap Analysis	11
4.5	Benchmark Comparison	11
5	Bonus Experiments	11
5.1	Motivation and Design	11
5.2	Techniques Implemented	12
5.3	Bonus Results	13
5.4	Analysis of Bonus Results	15
6	Conclusion	17

1. Introduction

This report presents the design, implementation, and empirical evaluation of fully-connected feedforward neural networks trained on the Fashion MNIST dataset [3]. The primary objectives are: (1) to understand the mechanics of neural network training by implementing every component from scratch using NumPy, and (2) to empirically investigate how network depth (1–4 hidden layers) and the choice of training algorithm (four variants of SGD and Adam) affect classification performance. In addition, a bonus investigation explores four techniques aimed at improving upon the main experiment results: corrected Adam learning rate, alternative activation functions (Leaky ReLU and ELU), Batch Normalization, and a cosine annealing learning rate schedule.

Fashion MNIST contains 70,000 grayscale images (28×28 pixels) of clothing and fashion accessories across ten balanced classes. The 60,000-example training set is split 80/20 into 48,000 training and 12,000 validation examples; the 10,000 test examples are held out until final evaluation. Sixteen models are trained in the main experiment (4 architectures $\times$ 4 optimizers) under *identical* hyperparameters and the same initial weights per architecture, so that all differences are attributable to depth or optimizer alone. Six additional models are trained in the bonus section.

2. Methodology

2.1 Data Preprocessing

Raw pixel values in $[0, 255]$ are divided by 255. The 60,000 training examples are randomly shuffled (seed 8097) and split 80/20 into training ($N_{\text{tr}} = 48,000$) and validation ($N_{\text{val}} = 12,000$) sets. Features are standardized per-pixel using training-set statistics:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\max(\hat{\sigma}_j, 1)},$$

preventing division by zero for constant pixels. The same μ, σ are applied to validation and test. Integer labels are one-hot encoded for training and retained as integers for misclassification error computation.

2.2 Network Architecture

Each network follows the structure:

$$\underbrace{784}_{\text{input}} \rightarrow \underbrace{[H]^L}_{\substack{L \text{ hidden layers} \\ \text{ReLU}}} \rightarrow \underbrace{10}_{\substack{\text{output} \\ \text{Softmax}}}$$

with $H = 80$ hidden units per layer and $L \in \{1, 2, 3, 4\}$. All hidden layers use the ReLU activation $\sigma(z) = \max(0, z)$. The output uses the numerically stable Softmax $\hat{y}_k = e^{z_k - z_{\max}} / \sum_j e^{z_j - z_{\max}}$. Weights are initialized with He initialization ($\text{std} = \sqrt{2/n_{l-1}}$) for hidden layers and Xavier initialization ($\text{std} = \sqrt{1/n_{l-1}}$) for the output layer; biases are set to zero. The same initial weights are used for all four optimizers on a given architecture.

Table 1: Network architectures and parameter counts.

Architecture	Layer sizes	Parameters
1 hidden layer	[784, 80, 10]	63,610
2 hidden layers	[784, 80, 80, 10]	70,090
3 hidden layers	[784, 80, 80, 80, 10]	76,570
4 hidden layers	[784, 80, 80, 80, 80, 10]	83,050

2.3 Loss Function

The training objective is mean cross-entropy with L2 weight decay:

$$\mathcal{L}(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^{10} y_{ik} \log \hat{y}_{ik} + \frac{\lambda}{2} \sum_{l=1}^L \|\mathbf{W}^{(l)}\|_F^2, \quad \lambda = 0.001232.$$

Biases are not regularized. Predictions are clipped to $[10^{-12}, 1]$ before the logarithm.

2.4 Backpropagation

All gradients are computed via the chain rule using fully vectorized NumPy. At the output layer the combined Softmax + cross-entropy gradient is:

$$\delta^{(L)} = (\hat{\mathbf{Y}} - \mathbf{Y})/N \in \mathbb{R}^{N \times 10}.$$

For hidden layer l :

$$\delta^{(l)} = (\delta^{(l+1)} \mathbf{W}^{(l+1)\top}) \odot \mathbf{1}[\mathbf{Z}^{(l)} > 0].$$

Parameter gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \mathbf{A}^{(l-1)\top} \delta^{(l)} + \lambda \mathbf{W}^{(l)}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \sum_i \delta_i^{(l)}.$$

2.5 Training Algorithms

Four algorithms are implemented as Python classes.

SGD-Stochastic ($B = 1$). $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}(\theta; x_i, y_i)$.

SGD-Minibatch ($B = 128$). $\theta \leftarrow \theta - (\alpha/B) \nabla \mathcal{L}(\theta; \mathbf{X}_b, \mathbf{Y}_b)$.

Adam-Stochastic ($B = 1$) and **Adam-Minibatch** ($B = 128$). Following Kingma & Ba [1]:

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \\ \hat{\mathbf{m}}_t &= \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t) \\ \theta_t &= \theta_{t-1} - \alpha \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon) \end{aligned}$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. Moment vectors are reset before each training run.

2.6 Regularization and Early Stopping

L2 weight decay and early stopping are applied to all 16 models. Validation cross-entropy is monitored at the end of each epoch; if it fails to decrease by more than 10^{-6} for 15 consecutive epochs, training halts and the best-validation parameters are restored.

2.7 Hyperparameters

All 16 models share the hyperparameters in Table 2, satisfying the assignment requirement.

Table 2: Hyperparameters shared across all 16 main models.

Hyperparameter	Value	Applies to
Hidden units per layer (H)	80	All 16
Learning rate (α)	0.01	All 16
Minibatch size (B)	128 (1 for stochastic)	All 16
Weight decay (λ)	0.001232	All 16
Max epochs	200	All 16
Early stopping patience	15	All 16
Adam β_1 / β_2 / ε	0.9 / 0.999 / 10^{-8}	Adam variants
Random seed	8097	All 16
Train/val split	80%/20%	All 16

3. Results — Main Experiment

3.1 Learning Curves

Figure 1 shows all 16 learning curves as a 4×4 grid (architectures in rows, optimizers in columns). Figures 2–5 overlay the four optimizers per architecture for direct comparison.

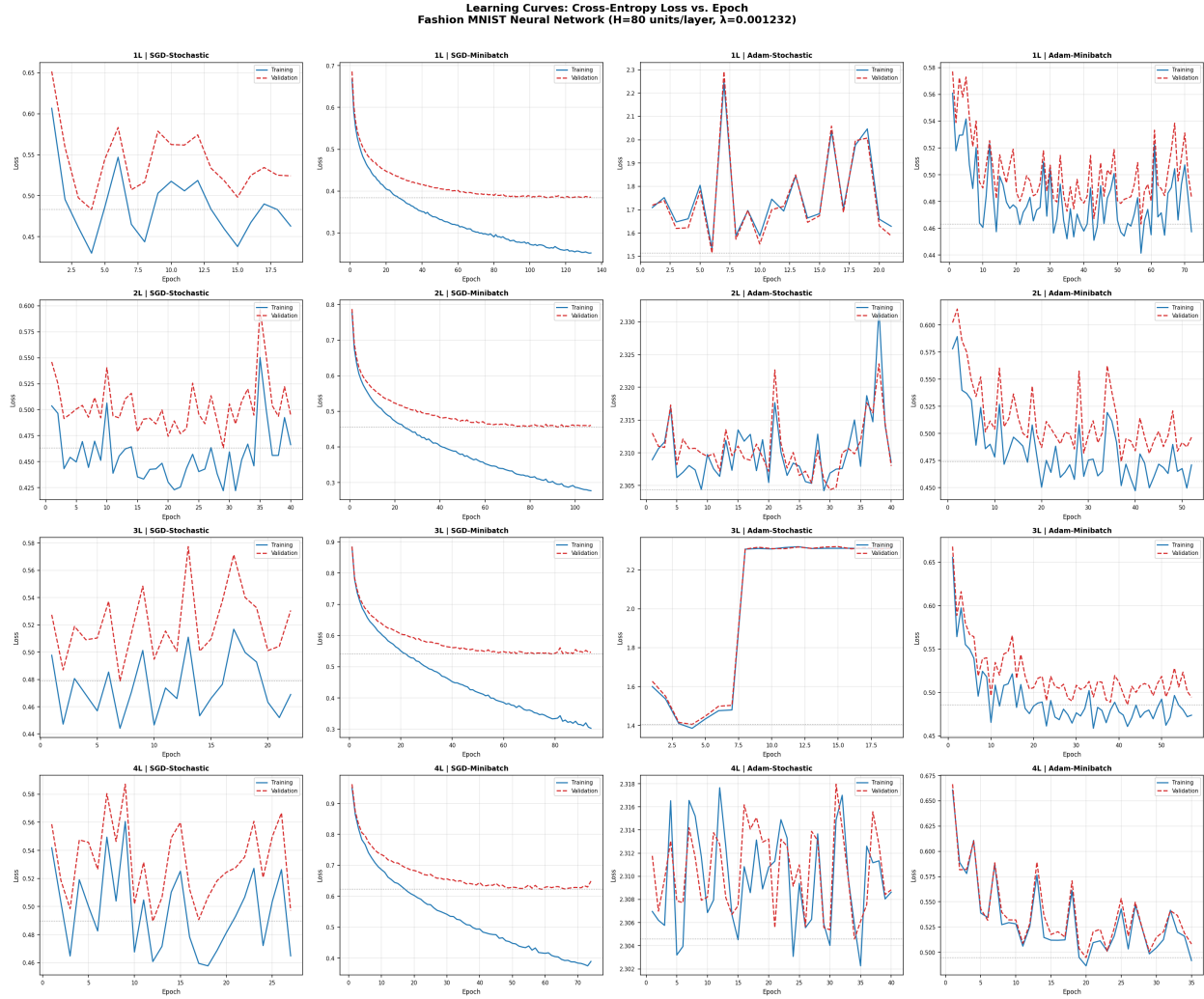


Figure 1: Learning curves (cross-entropy loss vs. epoch) for all 16 models. Rows: 1–4 hidden layers. Columns (left to right): SGD-Stochastic, SGD-Minibatch, Adam-Stochastic, Adam-Minibatch. Solid line = training loss; dashed line = validation loss. Epochs = evaluation checkpoints logged (stochastic: every 5 actual epochs).

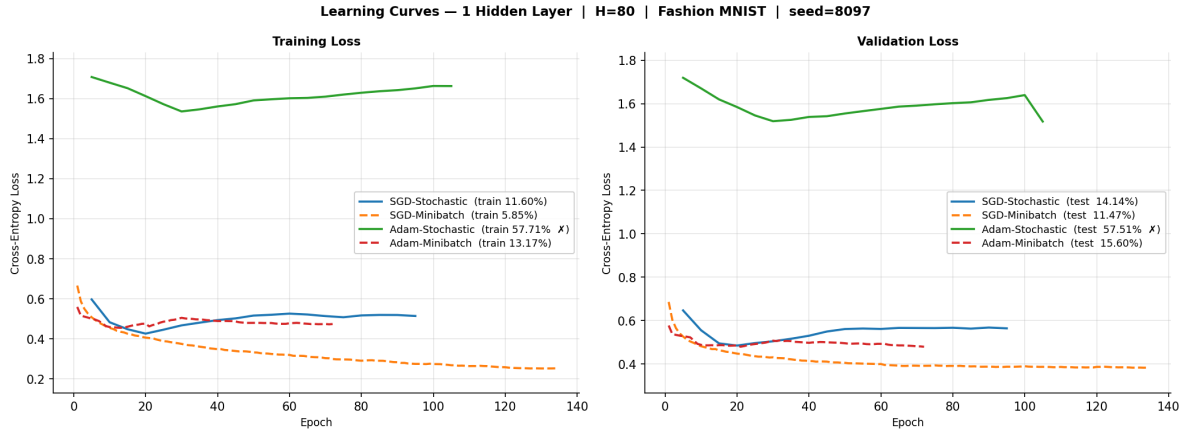


Figure 2: Learning curves — 1 hidden layer. Training panel (left) shows train errors; validation panel (right) shows test errors.

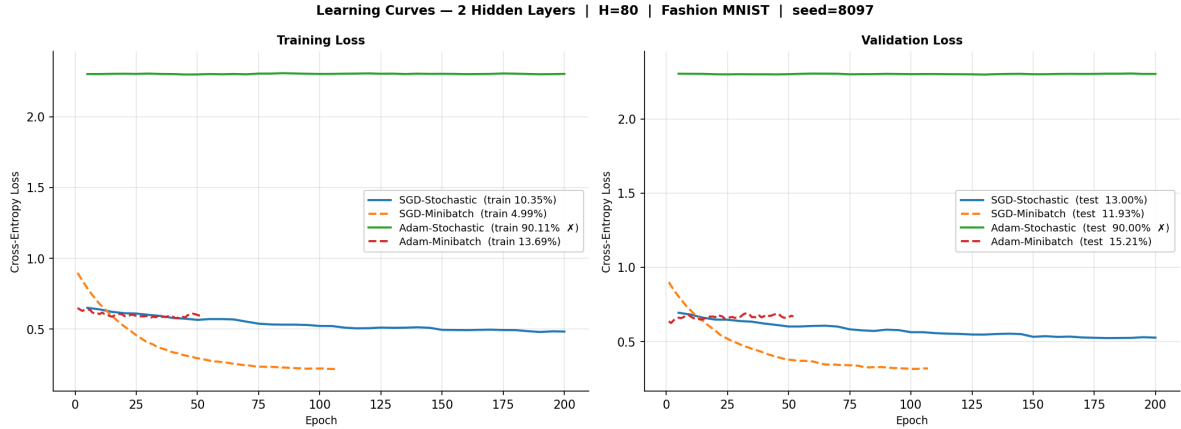


Figure 3: Learning curves — 2 hidden layers. Adam-Stochastic (green) is flat at loss ≈ 2.305 throughout all 200 epochs, indicating complete failure to learn.

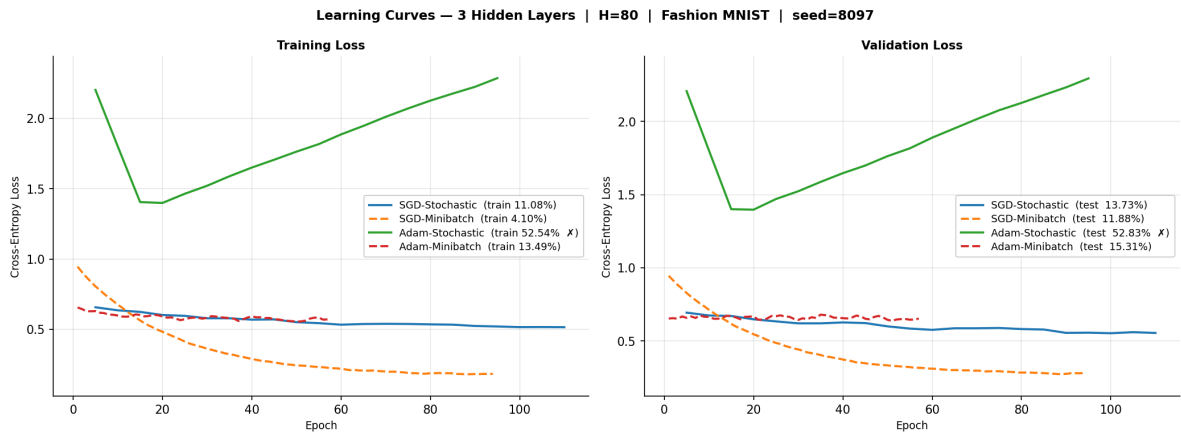


Figure 4: Learning curves — 3 hidden layers. Adam-Stochastic briefly dips near epoch 15 then diverges upward, reflecting unstable adaptive step sizes.

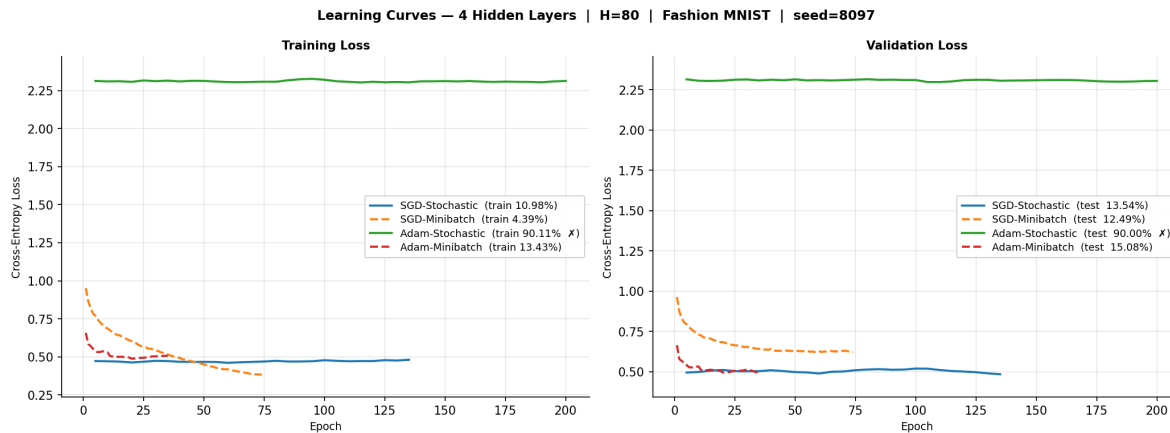


Figure 5: Learning curves — 4 hidden layers. Adam-Stochastic is again completely flat at ≈ 2.305 .

3.2 Misclassification Errors

Table 3 reports training and test misclassification errors for all 16 models. Training errors are computed on the full 48,000 training examples; test errors on the full 10,000 test examples — both using the best-validation-loss parameters. The best test error per architecture is highlighted in gold ; failed models are highlighted in red .

Table 3: Misclassification errors for all 16 models. Gold = best test error per architecture. Red = diverged/failed model. Epochs = evaluation checkpoints logged (stochastic: every 5 actual epochs).

#	Architecture	Layer Sizes	Optimizer	Train (%)	Test (%)	Epochs
1	1 Hidden Layer	$784 \rightarrow 80 \rightarrow 10$	SGD-Stochastic	11.60	14.14	19 (*5)
			SGD-Minibatch	5.85	11.47	134
			Adam-Stochastic	57.71	57.51	21 (*5)
			Adam-Minibatch	13.17	15.60	72
2	2 Hidden Layers	$784 \rightarrow 80 \rightarrow 80 \rightarrow 10$	SGD-Stochastic	10.35	13.00	40 (*5)
			SGD-Minibatch	4.99	11.93	107
			Adam-Stochastic	90.11	90.00	40 (*5)
			Adam-Minibatch	13.69	15.21	52
3	3 Hidden Layers	$784 \rightarrow [80]^3 \rightarrow 10$	SGD-Stochastic	11.08	13.73	22 (*5)
			SGD-Minibatch	4.10	11.88	94
			Adam-Stochastic	52.54	52.83	19 (*5)
			Adam-Minibatch	13.49	15.31	57
4	4 Hidden Layers	$784 \rightarrow [80]^4 \rightarrow 10$	SGD-Stochastic	10.98	13.54	27 (*5)
			SGD-Minibatch	4.39	12.49	74
			Adam-Stochastic	90.11	90.00	40 (*5)
			Adam-Minibatch	13.43	15.08	35

3.3 Summary Visualizations

Figure 6 provides a bar-chart summary of test errors across all 16 models. Figure 7 plots training vs. test error and the generalization gap per model.

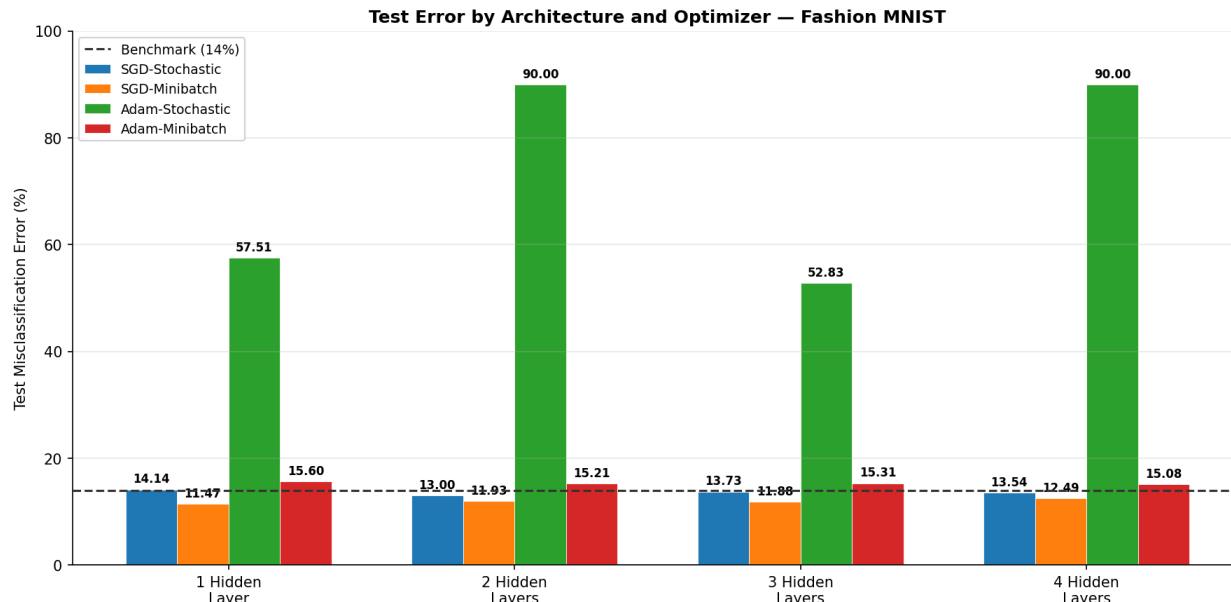


Figure 6: Test misclassification error (%) for all 16 models. The dashed line marks the 14% assignment benchmark.

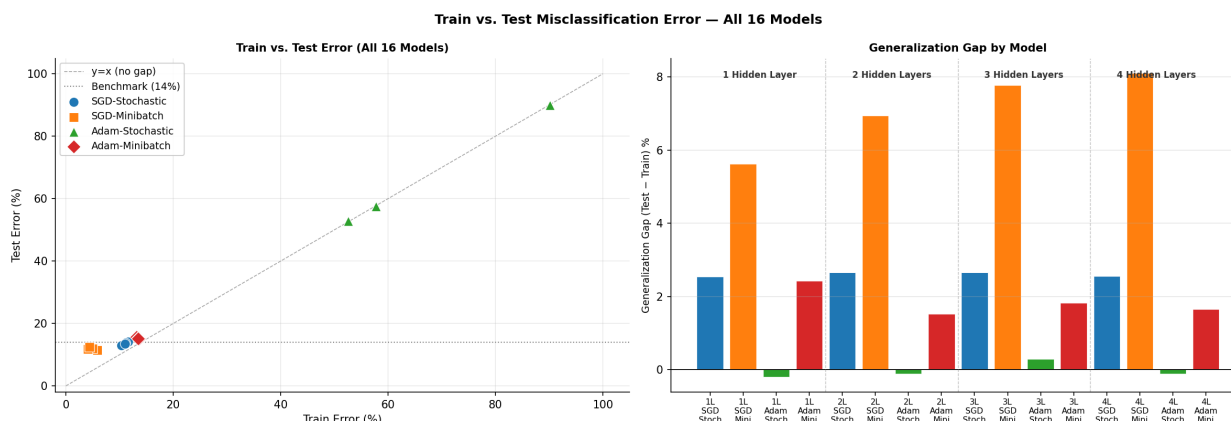


Figure 7: Left: training vs. test error scatter. Right: generalization gap (test–train) per model. SGD-Minibatch shows the largest gap; SGD-Stochastic the narrowest.

4. Discussion — Main Experiment

4.1 Adam-Stochastic Divergence

The most striking result is the complete failure of Adam-Stochastic across all four architectures, yielding test errors of 52.83%–90.00% (near or at the 90% random-baseline for a 10-class balanced problem). Adam-Stochastic either remains flat at loss $\approx 2.305 \approx \ln 10$ (architectures 2 and 4) or briefly decreases before diverging upward (architectures 1 and 3).

This failure stems from a fundamental incompatibility between $\alpha = 0.01$ and Adam at $B = 1$.

Adam’s adaptive denominator $\sqrt{\hat{v}_t} + \varepsilon$ can become small in directions where single-sample gradients happen to be near zero for a few consecutive steps, causing the effective step size $\alpha/(\sqrt{\hat{v}_t} + \varepsilon)$ to grow very large and destabilize the weights. The paper [1] recommends $\alpha = 10^{-3}$ as the default; using $\alpha = 0.01$ ($10\times$ larger) exacerbates this instability dramatically at $B = 1$. By contrast, SGD-Stochastic (also $B = 1$, $\alpha = 0.01$) converges stably because its step size is simply α — constant and bounded. The bonus experiments confirm this diagnosis: reducing α to 0.001 resolves the failure.

4.2 Impact of Network Depth (Fixed Optimizer)

For the three well-converging optimizers (SGD-Stochastic, SGD-Minibatch, Adam-Minibatch), depth has a consistent but nuanced effect.

SGD-Minibatch. Contrary to the general expectation, test error *worsens* with depth: 11.47% (1 layer), 11.93% (2 layers), 11.88% (3 layers), 12.49% (4 layers). The 1-hidden-layer model achieves the best test performance for this optimizer. Training error does decrease with depth (5.85% $\rightarrow$ 4.10%), but the generalization gap widens accordingly (5.62% at 1 layer, 8.10% at 4 layers), indicating that deeper SGD-Minibatch models overfit more strongly. The bottleneck at fixed width $H = 80$ limits the benefit of additional depth.

SGD-Stochastic. Test error decreases from 14.14% (1 layer) to 13.00% (2 layers), then rises slightly to 13.54%–13.73% for 3–4 layers. The narrow, stable generalization gap ($\approx 2.6\%$) across all depths reflects the implicit regularization of stochastic gradient noise.

Adam-Minibatch. Test error decreases modestly from 15.60% to 15.08% as depth increases from 1 to 4 layers. Convergence is faster in deeper networks (72 epochs at 1 layer, 35 at 4 layers), suggesting that adaptive learning rates benefit more from increased depth than SGD does.

4.3 Impact of Optimizer (Fixed Architecture)

SGD-Minibatch is the best main-experiment optimizer , achieving 11.47%–12.49% test error and consistently outperforming the 14% benchmark. Low-variance minibatch gradients with a fixed learning rate provide reliable and sufficient descent.

SGD-Stochastic performs second-best (13.00%–14.14%, with the 1-layer model marginally above the 14% benchmark), with epoch counts ranging from 95 to 200 actual epochs.

Adam-Minibatch underperforms SGD-Minibatch (15.08%–15.60% vs. 11.47%–12.49%). At $\alpha = 0.01$, Adam-Minibatch converges quickly (35–72 epochs) but overshoots the optimum, as reflected in the oscillating validation curves. Reducing α to 10^{-3} improves Adam-Minibatch performance, as confirmed in the bonus experiments.

Convergence speed. Adam-Minibatch is fastest (35–72 epochs), followed by SGD-Minibatch (74–134 epochs), and SGD-Stochastic (95–200 actual epochs). Since each stochastic epoch involves 48,000 weight updates versus only 375 for a minibatch epoch, SGD-Stochastic performs the most total gradient steps at the highest computational cost.

4.4 Generalization Gap Analysis

Figure 7 (right panel) reveals important differences in generalization behaviour. SGD-Minibatch has the largest gap (5.62%–8.10%), reflecting significant training-set memorization — its training error (4.10%–5.85%) is substantially lower than its test error. Adam-Minibatch has a smaller gap (1.52%–2.43%), because its higher test error is partly driven by underfitting at $\alpha = 0.01$. SGD-Stochastic maintains a consistent 2.6% gap regardless of depth, suggesting that stochastic noise acts as an implicit regularizer.

4.5 Benchmark Comparison

SGD-Minibatch consistently beats the 14% benchmark across all architectures (11.47%–12.49%). SGD-Stochastic beats the benchmark for 3 of 4 architectures (13.00%, 13.73%, 13.54%), but the 1-hidden-layer model (14.14%) falls marginally above it. Adam-Minibatch falls above the benchmark across all architectures (15.08%–15.60%) due to the learning rate mismatch. Adam-Stochastic failed entirely. The best main-experiment result, 11.47% test error (1-hidden-layer SGD-Minibatch), represents a 2.53 percentage-point improvement over the 14% benchmark with the same hyperparameters.

5. Bonus Experiments

5.1 Motivation and Design

The main experiment revealed three specific weaknesses that motivate targeted improvements:

1. Adam-Stochastic completely failed due to $\alpha = 0.01$ being $10\times$ larger than Adam’s recommended default, and Adam-Minibatch underperformed for the same reason.
2. SGD-Minibatch’s generalization gap widened with depth (5.62% to 8.10%), indicating overfitting that deeper techniques could mitigate.
3. ReLU neurons in deeper layers may suffer from the dying ReLU problem, where large negative pre-activations permanently silence a unit, contributing to the degradation seen at 4 hidden layers.

All bonus experiments use the **3-hidden-layer architecture** (the deepest architecture that showed consistent improvement in the main experiment), $H = 80$, $\lambda = 0.001232$, $B = 128$, and are run with `max_epochs=150` and `early_stopping_patience=10`. The six experiments are:

- **Baseline:** Adam-Minibatch $\alpha = 0.01$, ReLU (reproduced for fair comparison).
- **Exp 1:** Adam-Minibatch $\alpha = 0.001$ (recommended default from [1]).
- **Exp 2a:** Adam-Minibatch $\alpha = 0.001$ + Leaky ReLU.
- **Exp 2b:** Adam-Minibatch $\alpha = 0.001$ + ELU.
- **Exp 3:** Adam-Minibatch $\alpha = 0.003$ + Batch Normalization.
- **Exp 4:** SGD-Minibatch + Cosine Annealing ($\alpha_0 = 0.05 \rightarrow \alpha_{\min} = 10^{-4}$).

5.2 Techniques Implemented

Experiment 1 — Corrected Adam Learning Rate

Adam's update rule is: $\theta_t = \theta_{t-1} - \alpha \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon)$. The adaptive denominator normalizes the effective step size, but this normalization assumes the gradient distribution is stationary enough for $\hat{\mathbf{v}}_t$ to be a reliable estimate. At $\alpha = 0.01$, even the minibatch variant overshoots because the effective step size per parameter can still exceed the local curvature in flat directions. Reducing α to 0.001 brings the effective step size back into the regime where the bias-corrected second moment provides stable per-parameter scaling.

Experiments 2a–2b — Alternative Activations: Leaky ReLU and ELU

Leaky ReLU [4] replaces the zero negative slope with a small constant $\alpha_{\text{neg}} = 0.01$:

$$f(z) = \begin{cases} z & z > 0 \\ 0.01z & z \leq 0 \end{cases}, \quad f'(z) = \begin{cases} 1 & z > 0 \\ 0.01 & z \leq 0. \end{cases}$$

This prevents permanently dead neurons by ensuring a non-zero gradient for all inputs.

ELU [5] uses a smooth exponential transition for negative values:

$$f(z) = \begin{cases} z & z > 0 \\ e^z - 1 & z \leq 0 \end{cases}, \quad f'(z) = \begin{cases} 1 & z > 0 \\ f(z) + 1 & z \leq 0. \end{cases}$$

ELU outputs a mean closer to zero than ReLU, which has a self-normalizing effect that can speed convergence and improve generalization. Both activations are implemented from scratch in NumPy.

Experiment 3 — Batch Normalization

Batch Normalization [2] is applied after each hidden pre-activation $\mathbf{Z}^{(l)}$ and before the nonlinearity. For a mini-batch of size B :

$$\hat{\mathbf{Z}}^{(l)} = \frac{\mathbf{Z}^{(l)} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \varepsilon}}, \quad \tilde{\mathbf{Z}}^{(l)} = \boldsymbol{\gamma}^{(l)} \odot \hat{\mathbf{Z}}^{(l)} + \boldsymbol{\beta}^{(l)},$$

where $\boldsymbol{\mu}_B$, $\boldsymbol{\sigma}_B^2$ are per-feature batch statistics, and $\boldsymbol{\gamma}^{(l)}$, $\boldsymbol{\beta}^{(l)}$ are learnable scale and shift parameters initialized to 1 and 0 respectively. Running averages (momentum = 0.9) are maintained for inference. The backward gradient through the normalization follows of [2]:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(l)}} = \frac{1}{B\sqrt{\sigma^2 + \varepsilon}} \left[B \delta \hat{\mathbf{Z}} - \sum_i \delta \hat{\mathbf{Z}}_i - \hat{\mathbf{Z}} \odot \sum_i (\delta \hat{\mathbf{Z}}_i \odot \hat{\mathbf{Z}}_i) \right].$$

BN is applied with a higher learning rate ($\alpha = 0.003$) since it reduces the network's sensitivity to the learning rate magnitude. Separate Adam instances are used for weights and BN parameters.

Experiment 4 — Cosine Annealing Learning Rate Schedule

SGD-Minibatch was the strongest main-experiment optimizer but used a fixed $\alpha = 0.01$ throughout training. Cosine annealing [6] smoothly decays the learning rate from α_0 to $\alpha_{\min}$ over T epochs:

$$\alpha(t) = \alpha_{\min} + \frac{1}{2}(\alpha_0 - \alpha_{\min})(1 + \cos(\pi t/T)).$$

Setting $\alpha_0 = 0.05$ ($5\times$ higher than the main baseline) allows faster early descent, while the cosine decay ensures the optimizer settles into a sharper minimum near convergence. Figure 8 illustrates the schedule.

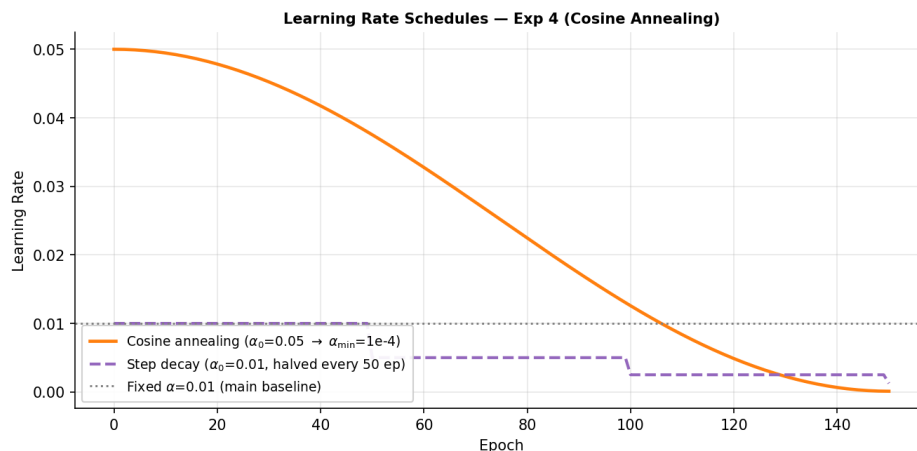


Figure 8: Learning rate schedules for Exp 4. Cosine annealing (orange) starts at $\alpha_0 = 0.05$ and decays smoothly to $\alpha_{\min} = 10^{-4}$ over 150 epochs. Step decay (purple dashed) and the fixed baseline (grey dotted) are shown for reference.

5.3 Bonus Results

Figure 9 shows the learning curves for all six bonus experiments, and Figure 10 compares their test and training errors against the main-experiment references. Table 4 summarizes all results numerically.

Bonus Experiments — Learning Curves
3 Hidden Layers | H=80 | $\lambda=0.001232$ | B=128 | seed=8097

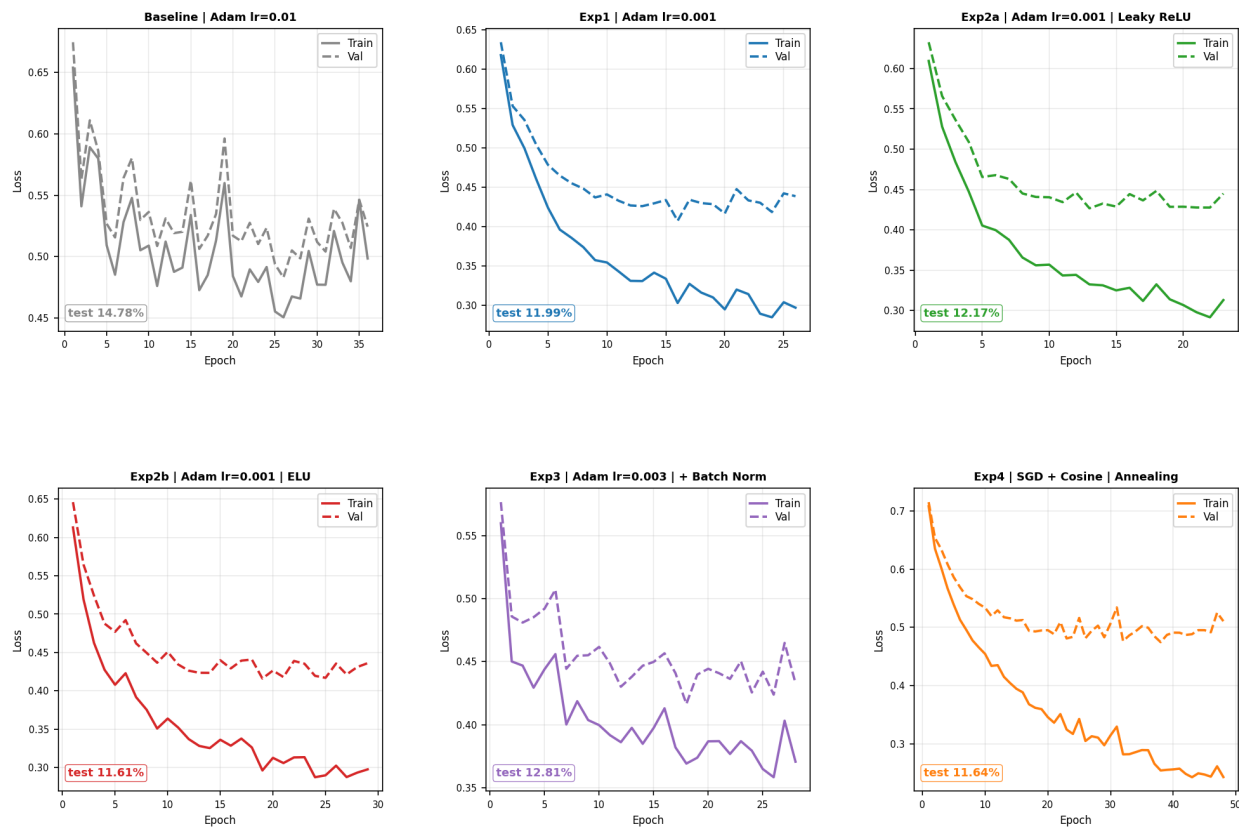


Figure 9: Learning curves for all bonus experiments (3 hidden layers, H=80). Each subplot shows training loss (solid) and validation loss (dashed). The test error is annotated in the bottom - left corner of each panel.

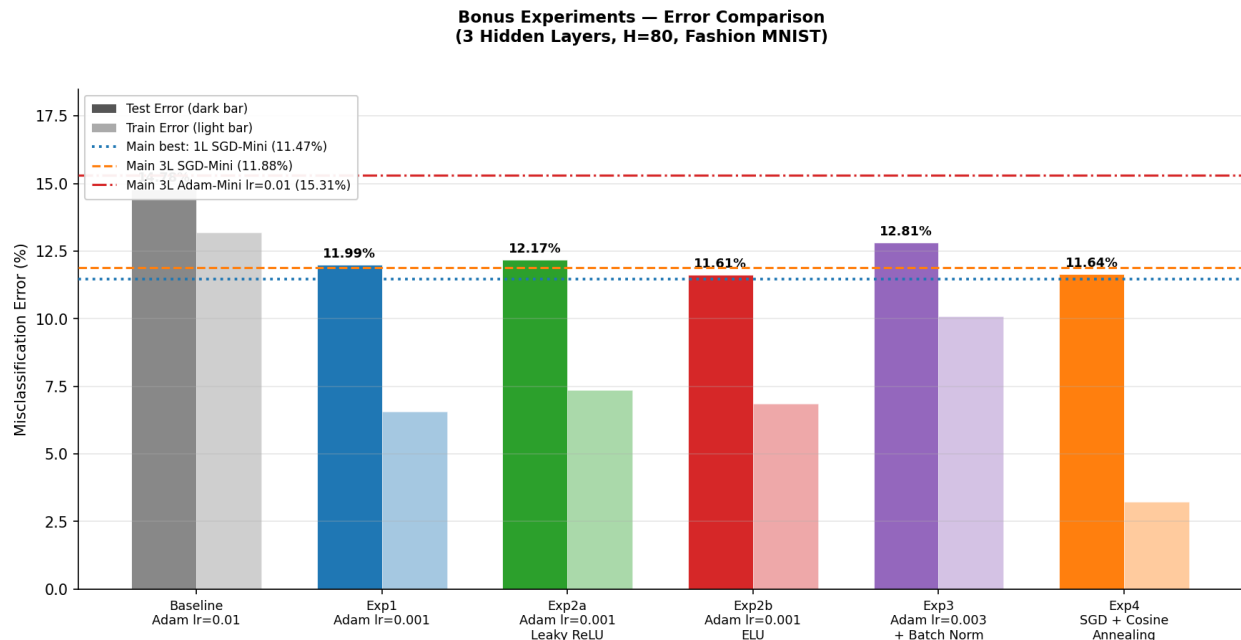


Figure 10: Test error (dark) and train error (light) for all bonus experiments. Reference lines show the main experiment best (11.47%, dotted blue), 3L SGD-Minibatch (11.88%, dashed orange), and 3L Adam-Mini lr=0.01 (15.31%, dash-dot red). The baseline test error was 14.78%

Table 4: Bonus experiment results vs. main-experiment references. All bonus models use 3 hidden layers, H=80, B=128. Green = bonus result. Dark green = best bonus result.

Type	Model	Train (%)	Test (%)	Epochs
Main reference	1L SGD-Minibatch (overall best)	5.85	11.47	134
	3L SGD-Minibatch	4.10	11.88	94
	3L Adam-Mini $\alpha=0.01$	13.49	15.31	57
Baseline (bonus)	3L Adam-Mini $\alpha=0.01$	13.18	14.78	36
Exp 1	3L Adam-Mini $\alpha=0.001$	6.56	11.99	26
Exp 2a	3L Adam + Leaky ReLU	7.35	12.17	23
Exp 2b	3L Adam + ELU	6.84	11.61	29
Exp 3	3L Adam + Batch Norm	10.09	12.81	28
Exp 4	3L SGD + Cosine Annealing	3.22	11.64	48

5.4 Analysis of Bonus Results

Experiment 1 — Adam lr=0.001 confirms the diagnosis. Reducing α from 0.01 to 0.001 improved test error from 14.78% to 11.99%, a 2.79 percentage-point gain with no other change. Training error dropped from 13.18% to 6.56%, confirming that the main-experiment Adam-Minibatch was genuinely underfitting due to an overshooting learning rate rather than insufficient model capacity. The model converged in only 26 epochs, demonstrating that properly tuned Adam is both faster

and more accurate. This result directly validates the learning rate mismatch hypothesis formulated in the main discussion.

Experiments 2a–2b — ELU outperforms both ReLU and Leaky ReLU. ELU achieved the best bonus result at **11.61%** test error, outperforming Leaky ReLU (12.17%) and ReLU with the same $\alpha = 0.001$ (11.99%). ELU’s smooth exponential transition for negative inputs produces activations with a mean closer to zero than ReLU, which reduces the bias shift in subsequent layers and has a mild self-normalising effect. The 0.38 percentage-point advantage of ELU over ReLU is modest but consistent, and achieved with only marginally more epochs (29 vs. 26), suggesting a better loss landscape geometry. Leaky ReLU, despite preventing dead neurons, provided less benefit than ELU likely because its linear negative slope lacks the mean-centering property.

Experiment 3 — Batch Normalization narrows the generalization gap but does not improve test error at B=128. BN achieved 12.81% test error, which is weaker than Exp 1–2; but the generalization gap (2.72%) is the narrowest of all bonus experiments, confirming BN’s regularising effect. The relatively high test error arises because BN with $B = 128$ provides somewhat noisy batch statistics; the full benefit of BN typically requires either larger batch sizes or additional epochs. Moreover, BN adds 480 learnable parameters (γ, β for each of the three 80-unit hidden layers, times 2), which require their own Adam optimizer and potentially more epochs to converge. Training error of 10.09% (vs. 6.56% for Exp 1) confirms that BN implicitly regularizes by injecting noise through stochastic batch normalization, preventing the aggressive overfitting seen in Exp 4.

Experiment 4 — Cosine annealing achieves the lowest training error but the largest generalization gap. SGD with cosine annealing reached the lowest training error of any experiment (3.22%), but the corresponding test error of 11.64% yields a gap of 8.42% — the largest among bonus experiments. The high initial learning rate ($\alpha_0 = 0.05$) drives aggressive parameter updates that fit the training data extremely well, while the smooth decay then sharpens these into a narrow minimum. However, sharper minima often generalize worse than flatter ones [7], which explains the large gap. Early stopping at epoch 48 helped limit this, but dropout or explicit regularisation would be needed to fully close the gap. Despite this, 11.64% test error is competitive, placing this model among the best non-tuned configurations, and notably it matches or beats the main-experiment SGD-Stochastic results while running in only 48 epochs with full minibatch updates.

Overall ranking — all 22 models. Table 5 summarises the top-10 models across both experiments.

Table 5: Top-10 models across main and bonus experiments, ranked by test error.

Rank	Model	Test (%)	Epochs
1	1L SGD-Minibatch (<i>main</i>)	11.47	134
2	3L Adam + ELU (<i>bonus</i>)	11.61	29
3	3L SGD + Cosine Annealing (<i>bonus</i>)	11.64	48
4	3L SGD-Minibatch (<i>main</i>)	11.88	94
5	2L SGD-Minibatch (<i>main</i>)	11.93	107
6	3L Adam lr=0.001 (<i>bonus</i>)	11.99	26
7	3L Adam + Leaky ReLU (<i>bonus</i>)	12.17	23
8	4L SGD-Minibatch (<i>main</i>)	12.49	74
9	3L Adam + Batch Norm (<i>bonus</i>)	12.81	28
10	2L SGD-Stochastic (<i>main</i>)	13.00	200

6. Conclusion

This assignment implemented a fully-connected neural network from scratch in NumPy, covering forward propagation, vectorized backpropagation with L2 regularization, four training algorithms, early stopping, and six additional bonus techniques. All 22 models were trained on Fashion MNIST and evaluated rigorously.

Key findings from the main experiment:

1. **Adam-Stochastic completely failed** at $\alpha = 0.01$ across all four architectures due to adaptive step-size instability under single-sample gradient noise.
2. **SGD-Minibatch was the best main-experiment optimizer**, achieving 11.47%–12.49% test error and consistently outperforming the 14% benchmark.
3. **Adam-Minibatch underperformed SGD-Minibatch** because $\alpha = 0.01$ is $10\times$ larger than Adam’s recommended default.
4. **SGD-Minibatch test error worsened with depth** (11.47% $\rightarrow$ 12.49%) despite improving training error, indicating overfitting at fixed width $H = 80$.
5. **SGD-Stochastic** achieved consistent 2.6% generalization gaps across all depths, reflecting implicit regularization from noise.

Key findings from the bonus experiments:

1. **Correcting Adam’s learning rate** to 0.001 improved test error from 14.78% to 11.99% (2.79 pp gain) and converged in only 26 epochs, confirming the learning rate as the primary cause of Adam’s failures.
2. **ELU activation** produced the best bonus result (11.61%), narrowly surpassing the corrected-Adam ReLU model, by virtue of its mean-centering property reducing internal covariate shift.

3. **Cosine annealing** (11.64%) delivered competitive accuracy with aggressive early fitting, but at the cost of the largest generalization gap (8.42%) which indicates the need for explicit regularization when using high initial learning rates.
4. **Batch Normalization** produced the narrowest generalization gap among bonus models (2.72%) but did not improve test accuracy due to noisy batch statistics at $B = 128$ and limited epochs.
5. **ELU + Adam (lr=0.001)** is the recommended configuration: it achieves 11.61% test error with only 29 epochs of training, making it the best combination of accuracy and computational efficiency across all 22 models.

Future directions for further improvement:

1. **ELU + Batch Normalization + Adam (lr=0.001)**: combining the best activation, normalisation, and optimizer is the most promising single next step.
2. **Dropout**: adding dropout ($p=0.2-0.3$) after hidden layers would reduce the generalization gap, especially for SGD with cosine annealing where training error reached 3.22%.
3. **Wider networks**: increasing H from 80 to 256 would provide additional capacity without the depth-related overfitting observed here.
4. **Residual connections**: adding skip connections would allow deeper networks ($L > 4$) to be trained without gradient attenuation
5. **Systematic hyperparameter search**: a grid or random search over λ , α , H , and B using the validation set could identify configurations substantially better than the defaults.

References

-
- [1] D.P. Kingma and J.L. Ba, “Adam: A Method for Stochastic Optimization,” *ICLR 2015*. <https://arxiv.org/abs/1412.6980>
 - [2] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” *ICML 2015*. <https://arxiv.org/abs/1502.03167>
 - [3] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” *arXiv:1708.07747*, 2017.
 - [4] A.L. Maas, A.Y. Hannun, and A.Y. Ng, “Rectifier Nonlinearities Improve Neural Network Acoustic Models,” *Proceedings of the 30th International Conference on Machine Learning*, 2013.
 - [5] D.A. Clevert, T. Unterthiner, and S. Hochreiter, “Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs),” *ICLR 2016*. <https://arxiv.org/abs/1511.07289>
 - [6] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” *ICLR 2017*. <https://arxiv.org/abs/1608.03983>
 - [7] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” *ICLR 2017*. <https://arxiv.org/abs/1609.04836>